1 Web Services Overview

1.1 Distributed Computing Overview

1.1.1 What is distributed computing?

Distributed computing is

- The method of making multiple computers work together to solve a common problem
  - Makes a computer network appear as a powerful single computer that provides large-scale resources to deal with complex challenges
- A model in which components of a software system are shared among multiple computers or nodes
  - Even though the software components may be spread out across multiple computers in multiple locations, they're run as one system
    - The systems on different networked computers communicate and coordinate by sending messages back and forth to achieve a defined task
- A type of computing in which different components and objects comprising an application can be located on different computers connected to a network

Key characteristics and Concepts

- Key Characteristics and Concepts
  - Decentralization: Nodes in a distributed system possess their own processing capabilities and memory
  - Networking: Devices communicate and coordinate actions via network protocols
  - Concurrent Processing: Tasks run in parallel across multiple nodes, accelerating performance
  - Fault Tolerance: If one node fails, the system continues to operate, often rerouting tasks to other nodes
  - Transparency: Users often perceive the system as a single entity rather than many independent components

Types of distributed architecture

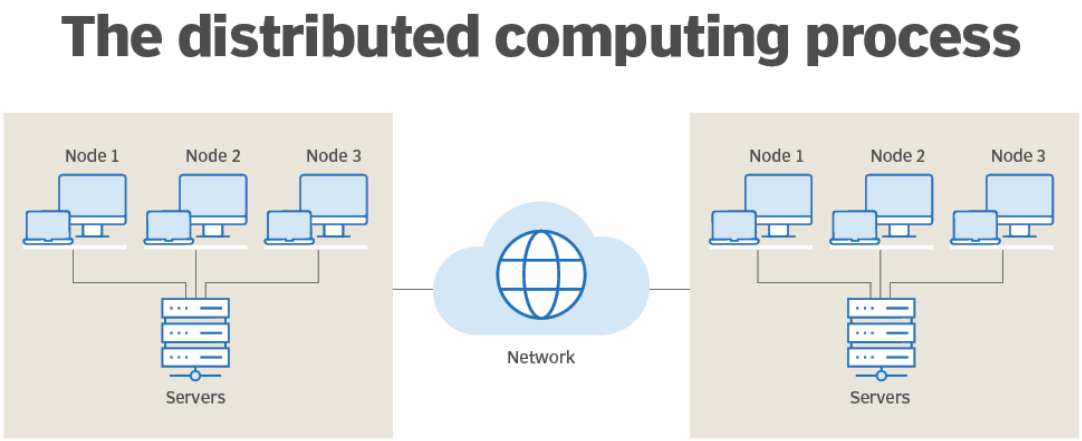
- Client-server architectures
  - Independent devices (clients) request services, data, or resources from a centralized, powerful computer (server)
    - Clients contact a server for data, then format and display that data to the user
- Peer-to-peer architectures

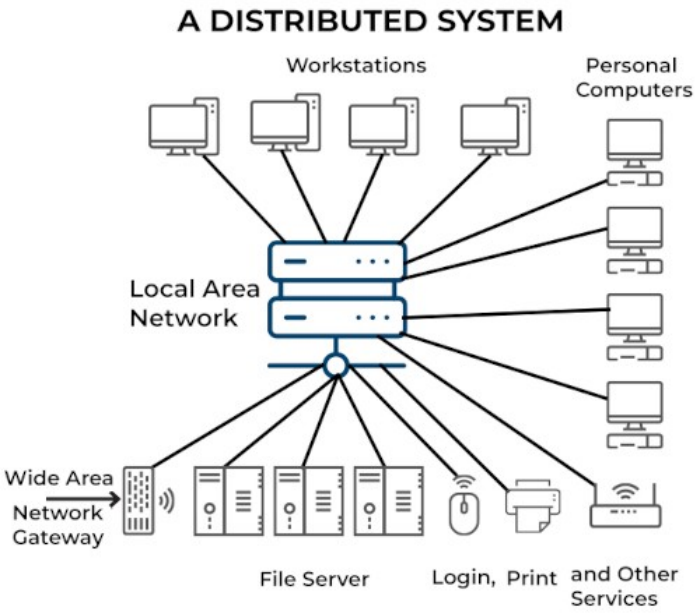
- Individual computers (peers) share resources – such as processing power, storage, or network capacity – directly with each other, bypassing a central server
  - Each node acts as both a client and a server

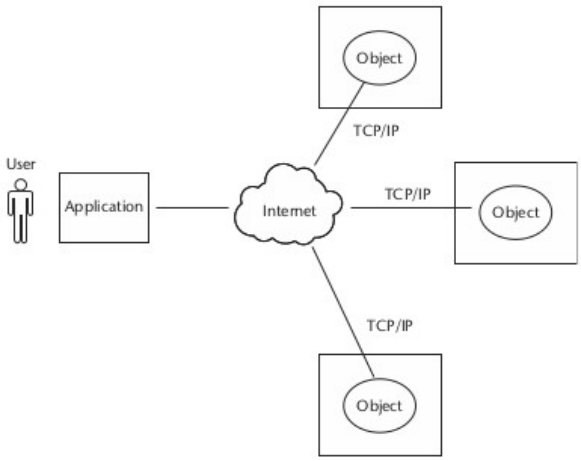
- Three-tier model
  - Software architecture that organizes applications into three distinct, physically separated layers: the presentation (client), application (logic), and data (database) tiers
    - This structure separates the user interface, business rules, and data management to improve scalability, security, and independent maintenance
      - User interface: occurs on the user's device at the user's location
      - Application processing takes place on a remote computer
      - Database access and processing algorithms happen on another computer that provides centralized access for many business processes

- N-tier system architectures
  - Typically used in application servers
  - Web applications forward requests to other enterprise services

Some illustrations





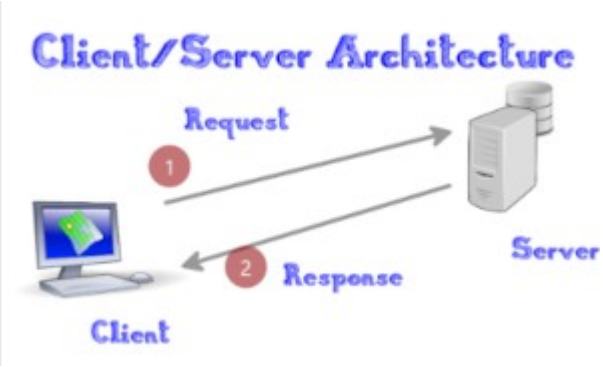


1.1.2 Benefits of distributed computing

- Performance/Efficiency: Each computer in a cluster handles different parts of a task simultaneously, i.e., applications can execute in parallel and distribute the load across multiple servers
- Scalability: New hardware are added when needed
- Extensibility/Collaboration
    - Dynamic (re)configuration of applications that are distributed across the network
        - Multiple applications can be connected through standard distributed computing mechanisms
- Resilience and redundancy
    - Multiple computers can provide the same services
    - If one machine isn't available, others can fill in for the service
    - If two machines that perform the same service are in different data centres and one data centre goes down, an organization can still operate
- Reuse/Resource sharing
    - The distributed components may perform various services that can potentially be used by multiple client applications
    - This saves repetitive development effort and improves interoperability between components
- Cost-effectiveness
    - Can use low-cost, off-the-shelf hardware
    - Reuse of developed components that are accessible over the network reduces cost
- Higher productivity and lower development cycle time
    - By breaking up large problems into smaller ones, these individual components can be developed by smaller development teams in isolation.

1.1.3 Review of distributed computing technologies

1.1.3.1 Client-Server



- Common in the early days of distributed computing
- Two-tier architecture model
  - Upper tier (Client): Handles the presentation and business logic of the user application
  - Lower tier (Server): Handles the application organization and its data storage (server)
- Popular design in business applications where the user interface and business logic are tightly coupled with a database server for handling data retrieval and processing.
- Model relies on Remote Procedure Calls (RPC) to execute code on a remote machine as if it were local
- Generally
  - Server is a database server that is mainly responsible for the organization and retrieval of data
  - The application client handles most of the business processing and provides the GUI of the application
- However
  - Model lacks the flexibility to scale across different programming languages and hardware

1.1.3.2 Distributed Object Models

These overcome the limitations of flat RPC

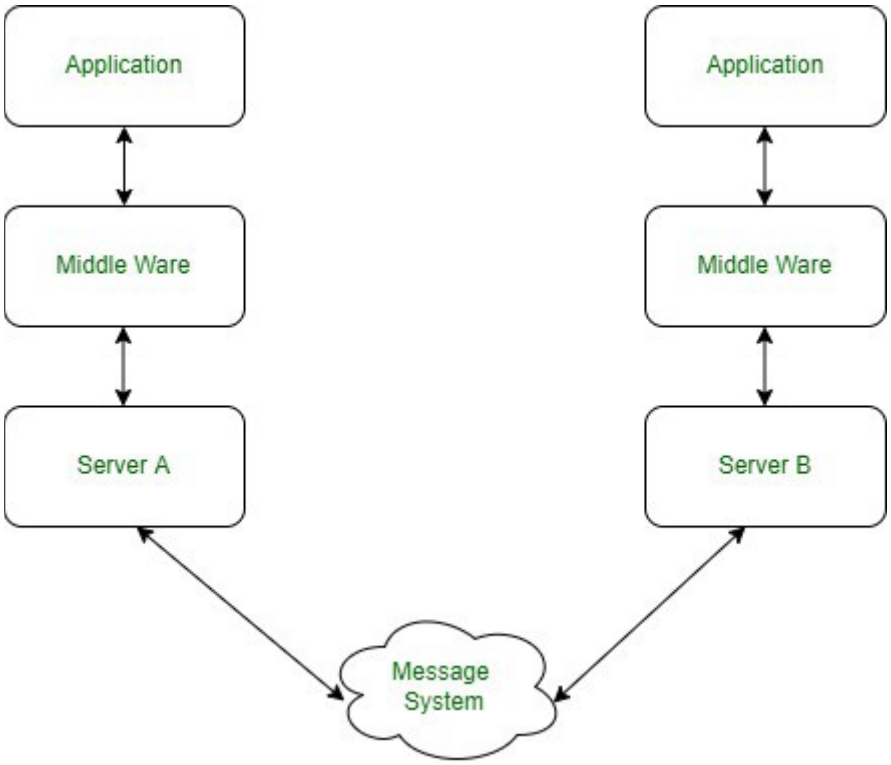
They treat remote resources as "objects," allowing for more modular and reusable code

Three major technologies: CORBA, DCOM, Java RMI

- **CORBA (Common Object Request Broker Architecture)**
  - A standard defined by the Object Management Group (OMG)
  - Key characteristic

        - Language-neutral
        - Designed to facilitate distributed computing that supports a wide range of application environments (different operating systems, programming languages, and computing hardware)
        - Allows a C++ client to talk to a Java server.
- **DCOM (Distributed Component Object Model)**
    - A Microsoft technology
    - Key characteristic
        - Platform-specific (optimized for Window-to-Windows communication)
        - Distributed computing platform that provides a way for Windows-based software components to communicate with each other
- Notes
    - DCOM works similarly to CORBA, but is tied to the Windows operating system whereas CORBA works on UNIX, Linux, SUN, OS X, and other UNIX-based platforms
    - Neither CORBA nor DCOM proved secure or scalable enough to become a standard for high volume web traffic, especially in the presence of firewalls
        - Big reason why HTTP became the default standard protocol for the internet
- **Java RMI**
    - Developed by Sun Microsystems
    - Key characteristic
        - Java-only; Offers deep integration with Java's garbage collection and security
        - Developed as the standard mechanism to enable distributed Java objects-based application development using the Java environment
        - RMI environment enables calling remote Java objects and passing them as arguments or return values
        - Difference with standard Java network libraries
            - java.net APIs were not intended to support or solve the distributed computing challenges
            - Java RMI uses Java Remote Method Protocol (JRMP) as the interprocess communication protocol, enabling Java objects living in different Java Virtual Machines (VMs) to transparently invoke one another's methods

1.2 Message-Oriented Middleware (MOM)



1.2.1 What is MOM?

- CORBA, RMI, and DCOM adopted a tightly coupled mechanism of a **synchronous** communication model (request/response)
- Tight integration across their logical tiers makes them not scale well
- Message-Oriented Middleware (MOM)
  - Based upon a loosely coupled **asynchronous** communication model
    - Distributed applications communicate asynchronously by exchanging messages, rather than direct function calls
    - Application client does not need to know its application recipients or its method arguments
    - Applications communicate indirectly using a messaging provider queue
      - Application client sends messages to the message queue (a message holding area), and the receiving application picks up the message from the queue
    - Sending application continues to operate without waiting for the response from that application

1.2.2 Benefits of MOM

- High Reliability: Messages are safely stored until the receiving component is ready to process them, ensuring no data loss
- Scalability: Facilitates the scaling of components independently
- Heterogeneous Integration: Enables communication between different operating systems and network protocols

1.2.3 Differences Between Message-Oriented Middleware (MOM) and Object-Oriented Distributed (OOD) Systems

	MOM		**OOD**
Communication Paradigm	Asynchronous message-passing via message queues The sender puts a message in a queue and proceeds with its own tasks without waiting for an immediate response.		Synchronous remote method invocation (RMI) or procedure calls (RPC) The client invokes a method on a remote object and must wait for the called procedure to return a result
Coupling	Loosely coupled The components do not need to be online at the same time and are insulated from each other's implementation details, making the system more resilient and scalable		Tightly coupled The client and the server must both be running and available for the interaction to occur, acting like a local function call
Abstraction	Provides the abstraction of a message queue (or a mailbox) Focus is on the data (the message) being passed, rather than the operation itself		Provides the abstraction of a remote object with methods that can be invoked Focus is on the invocation of operations and making remote objects appear local.
Reliability	Inherently reliable, as messages are stored in persistent queues until successfully delivered and processed, ensuring no message is lost even if the receiving application is temporarily unavailable		Requires robust error handling for network failures or application downtime, as synchronous calls can fail if the remote object is unreachable
Scalability & Performance	Generally highly scalable and offers potential for parallelism, as the asynchronous nature allows multiple producers and consumers to interact with queues independently		Less potential for parallelism in a single thread without using multiple threads because of the blocking nature of synchronous calls.

1.3 Common Challenges in Distributed Computing

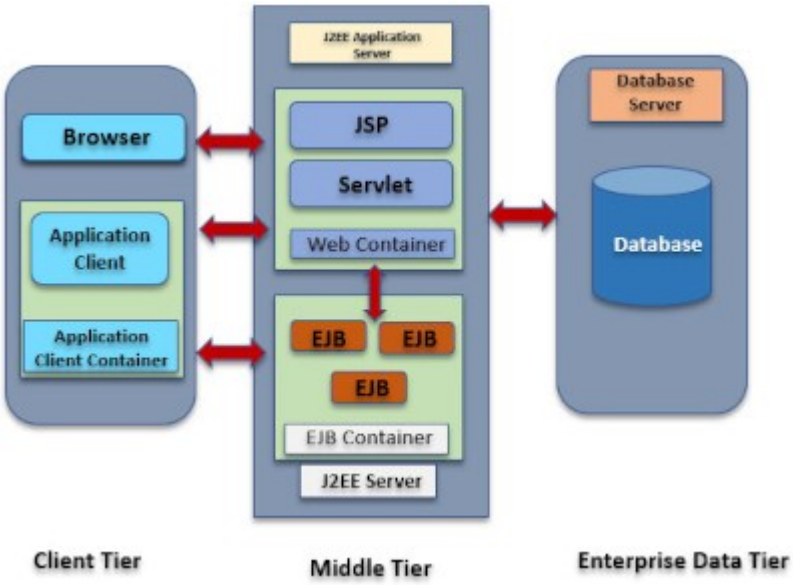
- Distributed computing technologies like CORBA, RMI, and DCOM have been quite successful in integrating applications within a homogenous environment inside a local area network
- But the Internet (interconnection of networks) demands the interoperability of applications across networks
- Common challenges in these distributed computing solutions
  - Maintenance of various versions of stubs/skeletons in the client and server environments is extremely complex in a heterogeneous network environment
  - Lots of application development time spent for Quality of Service (QoS) goals: Scalability, Performance, and Availability in a distributed environment
  - Interoperability of applications implementing different protocols on heterogeneous platforms is almost impossible, e.g., a DCOM client communicating to an RMI server or an RMI client communicating to a DCOM server
  - Protocols used are designed to work well within local networks; not very firewall friendly or able to be accessed over the Internet

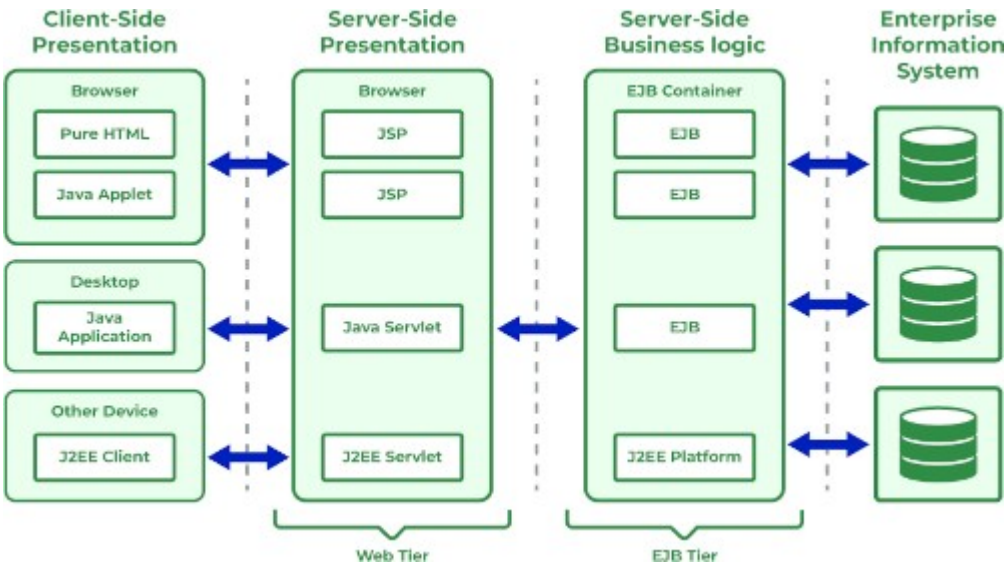
1.4 The Role of J2EE and XML in Distributed Computing

- Internet has come to enable enterprise applications
  - Easily accessible over the Web without having specific client-side software installations
- Initial focus was to move the complex business processing toward centralized servers in the back end
- First generation of Internet servers
  - Based upon Web servers
  - Web servers hosted static web pages and provided content to the clients via HTTP (HyperText Transfer Protocol), a stateless protocol that connects web browsers to web servers, enabling the transportation of HTML content to the user
- Server-side scripting
  - Motivated by need for a more dynamic technology
- Business-to-business (B2B) and business-to-consumer (B2C) models
  - Motivated by organizations moving their businesses to the Internet
  - This evolution led to the specification of J2EE architecture, which promoted a much more efficient platform for hosting Web-based applications
  - J2EE provides a programming model based upon Web and business components that are managed by the J2EE application server

- The application server consists of many APIs and low-level services available to the components
  - These low-level services provide security, transactions, connections and instance pooling, and concurrency services, which enable a J2EE developer to focus primarily on business logic rather than plumbing

J2EE architecture





Three logical tiers, with the various application components having defined roles and responsibilities

- Presentation (Client tier)
  - Consists of programs or applications that interact with the user, usually, located on a different machine from the server
  - Handles HTTP requests/responses, session management, device independent content delivery, etc.
  - A client can be a web browser, stand-alone application, or server on a different machine.
- Middle Tier (Web tier & EJB Tier)
  - Also known as the application tier or the business tier
  - Deals with the core business logic processing, i.e., workflow and automation
  - Web Tier/Web Component
    - Can be servlets or JSP pages
    - Servlets can dynamically process the request and generate the responses
    - JSP pages on the other hand, are static
  - EJB Tier/EJB Component
    - EJB components for processing user inputs and sending the input to Enterprise Bean with the business logic code
    - If necessary, Enterprise Container receives data from client processes and sends it to the enterprise information system for storage. Enterprise Bean also retrieves data from storage, processes it, and sends it back to the client.
- Enterprise data tier
  - Comprises database servers, enterprise resource planning systems, and other data sources
  - Resources are typically located on a separate machine from the J2EE Server and accessed by components on the business tier

XML (Extensible Markup Language)

- Used as a communication medium for electronic data exchange
- Used for defining portable data in a structured and self-describing format
- Promotes interoperability between applications and also enhances the scalability of the underlying applications
  - e.g., application-to-application communication on the Internet
- XML also promotes Web services technology

1.5 Emergence of Web Services

- Motivation
  - Discrete internet-based business applications, which co-exist in the same technology space but without interacting with each other
  - Industry's need for enabling B2B, application-to-application (A2A), and inter-process application communication led to a requirement for service-oriented architectures (SOAs)
  - Service-oriented applications
    - Expose business applications as service components
    - This enables business applications from other organizations to link with these services for application interaction and data sharing without human intervention
    - SOA facilitate the delivery of services over the Internet by leveraging standard technologies like XML and using platform-neutral standards, making the application components available to any application, any platform, or any device, and at any location
    - This is what web services are all about.